

Escaping ULM Mode

Getting VMware Workstation out of user-level monitor mode so nested virtualization actually works — and finding which invisible Windows feature is holding the hypervisor hostage.

Verified 2026-08-16

Updated 2026-10-07

Workstation 25.0.0

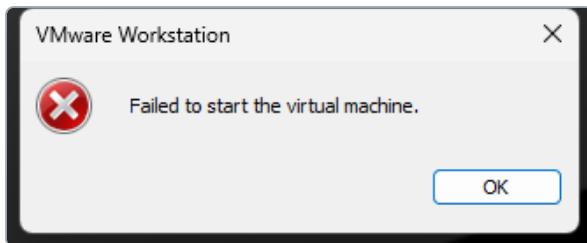
Host Win 11 Pro 26200

Reference box Ryzen 7 5800X

Scripts on GitHub

01 ROOT CAUSE

The symptom is a dialog with no diagnostic value: **"Failed to start the virtual machine"**, appearing only once *Virtualize Intel VT-x/EPT or AMD-V/RVI* is ticked. The real chain is five links long, and only the last one is visible.



The entire diagnostic Windows offers. No error code, no log reference, nothing pointing at the host.

● A VBS consumer is enabled in Windows

Something asks for Virtualization-Based Security. On the reference machine this was **Windows Hello Enhanced Sign-in Security (ESS)** — not Hyper-V, not Memory Integrity.

● Windows loads its own hypervisor at boot

`hvax64.exe` on AMD, `hvik64.exe` on Intel. Both ship with the OS whether or not the Hyper-V role is installed, so no feature needs to be enabled for this to happen.

● The hypervisor claims the CPU's virtualization extensions

It now owns AMD-V/RVI or VT-x/EPT exclusively. Windows starts reporting `HypervisorPresent: True`, and — the useful tell — `VMMonitorModeExtensions` and `SecondLevelAddressTranslationExtensions` both flip to `False`, because they are masked from the OS.

● Workstation demotes itself to ULM

Rather than fail, it runs as a **user-level monitor** on the Windows Hypervisor Platform instead of using its own ring-0 monitor (`vmx86`). Ordinary VMs still boot fine, which is exactly why this hides so well.

● Nested virtualization is unavailable in ULM

A user-level monitor cannot pass the virtualization extensions down to a guest. Setting `vhv.enable` makes power-on fail — with that generic dialog and nothing useful in the UI.

THE ONE LINE THAT PROVES IT

Every VM writes its monitor mode to its own `vmware.log`, in the VM's folder. This is the fastest confirmation, before and after:

VMWARE.LOG

BEFORE FIX

```
2026-08-15T21:08:49.317Z vmware-vmx.exe Monitor Mode: ULM
```

WHY THE USUAL ADVICE FAILS HERE

Every guide online says to disable Hyper-V and turn off Memory Integrity. On the reference machine **all of that was already off** — every Hyper-V optional feature Disabled, HVCI `0`, Credential Guard `0`, `hvhost` stopped and disabled — and the hypervisor still ran. ESS is VBS-backed but appears in *neither* the Hyper-V feature list *nor* the Core Isolation UI. It is only visible in the registry.

02 BEFORE YOU TOUCH THE OTHER LAPTOP

DO NOT BLIND-COPY THE TWO COMMANDS

The fix that worked on the desktop was aimed at a cause that machine happened to have. A laptop is a materially different environment, and three things commonly differ:

- **BitLocker is usually ON.** Windows 11 laptops enable device encryption by default; the desktop did not have it. Editing the BCD on an encrypted machine can trigger a recovery-key prompt at next boot. Get the key *first*.
- **Disabling ESS destroys the Windows Hello enrolment — PIN included.** Hello's credentials sit in a VBS-sealed container; remove VBS and the whole container becomes unreadable, so PIN, face and fingerprint all go "currently unavailable" at once and `dsregcmd /status` reports `NgcSet : NO`. You are not locked out — the account password still works — but you must re-enrol the PIN afterwards. Know the password before you start.
- **WSL2, Docker Desktop or a QEMU-backed app may be installed.** If so, do not disable the hypervisor globally — use the dual-boot entry in step 06 instead.

Run the diagnosis in step 03 and let the results pick the fix.

A Secure the recovery key

In an **elevated** PowerShell. If protection status is On, retrieve the recovery key from `account.microsoft.com/devices/recoverykey` and have it on your phone before continuing.

```
POWERSHELL (ADMIN)
```

```
READ-ONLY
```

```
manage-bde -status
```

B Read the full hypervisor picture

This is read-only and needs no elevation. It reports every mechanism that can load the hypervisor, in one pass.

```
POWERSHELL READ-ONLY

# 1 - is a hypervisor running, and are the CPU extensions masked?
Get-CimInstance Win32_ComputerSystem | Select-Object HypervisorPresent
Get-CimInstance Win32_Processor | Select-Object Name, VMMonitorModeExtensions,
    SecondLevelAddressTranslationExtensions, VirtualizationFirmwareEnabled

# 2 - VBS running state
Get-CimInstance -Namespace root\Microsoft\Windows\DeviceGuard `
    -ClassName Win32_DeviceGuard |
    Select-Object VirtualizationBasedSecurityStatus,
        @{n='Running';e={$_.SecurityServicesRunning -join ', '}}

# 3 - EVERY VBS scenario, not just Memory Integrity. This is the one
#     that other guides omit, and where ESS shows up.
Get-ChildItem 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios' |
    ForEach-Object {
        "--- $($_.PSChildName)"
        (Get-ItemProperty $_.PSPath |
            Select-Object * -Exclude PS* | Format-List | Out-String).Trim()
    }

# 4 - Hyper-V features, via WMI (see gotcha below)
Get-CimInstance Win32_OptionalFeature |
    Where-Object { $_.Name -match 'Hyper-V|VirtualMachinePlatform|HypervisorPlatform|DisposableClientVM'
} |
    Select-Object Name, @{n='State';e={switch($_.InstallState){1{'Enabled'}2{'Disabled'}3{'Absent'}}}}
```

GOTCHA - CLASS NOT REGISTERED

`Get-WindowsOptionalFeature -Online` fails with `Class not registered` under PowerShell 7, because the DISM module's COM interface doesn't load there. Use the `Win32_OptionalFeature` query above instead, or run that one cmdlet inside Windows PowerShell 5.1. It is not a sign of a broken machine.

NAME THE CULPRIT, DON'T GUESS IT

If a feature came back enabled and you don't know what did it, the DISM log records the exact command line and timestamp. This is how Try Omarchy was caught in one grep:

```
POWERSHELL READ-ONLY

Select-String -Path 'C:\Windows\Logs\DISM\dism.log' `
    -Pattern 'enable-feature' | Select-Object -Last 10
```

C Confirm the BCD setting

Elevated — `bcdedit` silently produces nothing without it, which reads as an empty result rather than an error.

POWERSHELL (ADMIN)

READ-ONLY

```
bcdedit /enum "{current}" | findstr /i hypervisorlaunchtype
```

Enabling a hypervisor feature resets this to `Auto`, silently overriding an earlier `off`. Always re-read it after any app has touched the feature list.

04 PICK THE FIX FROM WHAT YOU FOUND

Work down this table and stop at the first row that matches. More than one can be true; fix them in order and re-check.

FINDING	MEANING	ACTION
HypervisorPresent False	No hypervisor. This is not your problem.	Check firmware instead — VirtualizationFirmwareEnabled must be True, else enable SVM (AMD) or VT-x (Intel) in BIOS.
HypervisorPlatform Enabled and nothing else	A QEMU-backed app turned WHP on for itself. Try Omarchy, QEMU, VirtualBox 7 and the Android Studio emulator all use the Windows Hypervisor Platform as their acceleration path, and their installers enable it silently. Try Omarchy 0.9.0 runs dism /online /enable-feature /featurename:HypervisorPlatform /all /norestart /quiet and drops a whp-requested receipt in its install folder.	Use step 06, not 05a. The app re-enables WHP the next time it launches and finds it gone, so disabling the feature starts a fight you lose on every reboot. A dedicated boot entry keeps both working.
Any other Hyper-V feature Enabled	Hyper-V, WSL2 or Sandbox is installed and owns the CPU.	Disable the features (step 05a), <i>then</i> apply 05c.
HypervisorEnforcedCodeIntegrity = 1	Memory Integrity is on.	Windows Security → Device security → Core isolation → off. Reboot, re-check.
WindowsHello = 1 all other scenarios 0	The reference-machine case. ESS alone is holding the hypervisor up.	Apply 05b + 05c together.
All scenarios 0, no features enabled	The BCD is launching the hypervisor on its own.	05c alone is enough.

All elevated. Do not reboot until you have run every step that applies.

a Suspend BitLocker, then remove Hyper-V features

Skip the feature block entirely if step 03 showed everything already Disabled — it was a no-op on the reference machine.

```
POWERSHELL (ADMIN) MODIFIES SYSTEM

Suspend-BitLocker -MountPoint "C:" -RebootCount 1

'Microsoft-Hyper-V-All','HypervisorPlatform','VirtualMachinePlatform',
'Containers-DisposableClientVM','Windows-Defender-ApplicationGuard' | ForEach-Object {
    try { Disable-WindowsOptionalFeature -Online -FeatureName $_ -NoRestart -ErrorAction Stop | Out-Null
        "disabled: $_" }
    catch { "skipped: $_" }
}
```

b Turn off the ESS scenario

Only if step 03 showed `WindowsHello\Enabled = 1`. This is the step no other guide includes.

```
POWERSHELL (ADMIN) MODIFIES REGISTRY

Set-ItemProperty `
    -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\WindowsHello' `
    -Name Enabled -Type DWord -Value 0
```

c Stop the hypervisor from launching, then reboot

This is the load-bearing command. It prevents the hypervisor from starting regardless of which consumer asked for it.

```
POWERSHELL (ADMIN) MODIFIES BOOT CONFIG

bcdedit /set hypervisorlaunchtype off
bcdedit /set vsmlaunchtype Off

Restart-Computer
```

d Re-enrol Windows Hello after the reboot

Expect this: every Hello option reads *"This option is currently unavailable"* and `dsregcmd /status` shows `NgcSet : NO`. The old container was sealed to VBS and cannot be opened now that VBS is gone. Sign in with your account password, then clear the stale container so Windows can build a fresh one.

BEFORE YOU RUN THIS

Confirm you know the account password — it is the only credential that still works. On a Microsoft account, verify it at `account.microsoft.com` from your phone *first*. Clearing the container removes Hello enrolments for every user on the machine.

POWERSHELL (ADMIN)

DELETES CREDENTIALS

```
# take ownership of the Hello container, then empty it
$ngc = "$env:WinDir\ServiceProfiles\LocalService\AppData\Local\Microsoft\Ngc"
takeown /f $ngc /r /d y
icacls $ngc /grant "$($env:USERNAME):F" /t
Remove-Item "$ngc\*" -Recurse -Force

# hand ownership back so Windows can manage it again
icacls $ngc /t /q /c /reset

Restart-Computer
```

After the reboot: **Settings** → **Accounts** → **Sign-in options** → **PIN (Windows Hello)** → **Set up**. The new PIN is enrolled without ESS, so it survives the hypervisor staying off. Re-enrol face and fingerprint from the same page if the machine has them. Confirm with `dsregcmd /status` — `NgcSet` should read `YES`.

Disabling the hypervisor globally breaks WSL2, Docker Desktop's WSL2 and Hyper-V backends, Windows Sandbox, and any QEMU-backed app that accelerates through WHP. If the machine uses any of them, add a second boot entry instead of flipping the setting back and forth.

POWERSHELL (ADMIN)

MODIFIES BOOT CONFIG

```
# copy the current entry; this prints a GUID
bcdedit /copy "{current}" /d "Windows 11 (No Hyper-V)"

# apply the setting to THAT entry only, using the GUID printed above
bcdedit /set "{PASTE-GUID-HERE}" hypervisorlaunchtype off
```

You then pick *Windows 11 (No Hyper-V)* at boot for nested virtualization work, and the normal entry for containers and QEMU guests.

WHY THIS BEATS TOGGLING THE FEATURE

An app that enabled WHP for itself will simply enable it again the next time it starts and finds it missing — it never asks. Removing the feature therefore buys one reboot, not a fixed machine. Leaving the feature installed and controlling the *boot entry* takes the decision back without the app noticing anything is wrong.

After the reboot, all four of these must agree. Two of the four inverting is the signal that the extensions were released.

POWERSHELL

READ-ONLY

```
Get-CimInstance Win32_ComputerSystem | Select-Object HypervisorPresent
Get-CimInstance Win32_Processor |
  Select-Object VMMonitorModeExtensions, SecondLevelAddressTranslationExtensions
Get-CimInstance -Namespace root\Microsoft\Windows\DeviceGuard `
  -ClassName Win32_DeviceGuard | Select-Object VirtualizationBasedSecurityStatus
```

SIGNAL	BROKEN	FIXED
HypervisorPresent	True	False
VMMonitorModeExtensions	False	True
SecondLevelAddressTranslationExtensions	False	True
VirtualizationBasedSecurityStatus	2 running	0 not enabled

EXPECTED LEFTOVERS – NOT A FAILURE

The `hvservice` and `Vid` drivers may still show as Running after the fix. They load regardless and have no hypervisor to attach to. The CPUID and VBS results above are authoritative; ignore the driver states.

→ Then prove it end to end

Power the VM **fully off** — not suspended; nested virtualization cannot be toggled from a suspended state. In VM settings → Processors, tick *Virtualize Intel VT-x/EPT* or *AMD-V/RVI* to match the host CPU. That writes `vhv.enable = "TRUE"` into the `.vmx`. Start the VM, then read the log back:

POWERSHELL

READ-ONLY

```
Select-String -Path 'F:\VMs\*\vmware.log' -Pattern 'Monitor Mode' |
  Select-Object -Last 3
```

CHECK THE TIMESTAMP BEFORE YOU TRUST THE LINE

A log only records the mode from the run that produced it. After a reboot, every existing log still says `ULM` until you actually start a VM again — which looks exactly like a failed fix. Compare the log's write time against `(Get-CimInstance Win32_OperatingSystem).LastBootUpTime`; anything older is stale. You want `Monitor Mode: CPL0` from a run *after* the reboot.

08 ROLLBACK

Fully reversible, and worth knowing before you start. Elevated, then reboot:

```
POWERSHELL (ADMIN) RESTORES DEFAULTS

bcdedit /set hypervisorlaunchtype auto

Set-ItemProperty `
  -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\WindowsHello' `
  -Name Enabled -Type DWord -Value 1
```

Re-enable any optional features from step 05a through *Turn Windows features on or off*, and Memory Integrity through Windows Security.

- **The entire Windows Hello enrolment, PIN included.** Not just biometrics — the PIN goes too, because all three share one VBS-sealed container. Recovery is step 05d: clear the container and enrol a new PIN, which then works without ESS. Only the account password survives the change untouched, so have it to hand.
 - **WSL2, Docker Desktop, Windows Sandbox, Hyper-V VMs, QEMU-backed apps.** All unavailable while the hypervisor is off — unless you use the dual-boot entry in step 06.
 - **Feature updates can undo this.** A major Windows update may re-enable ESS. If nested virtualization breaks again months from now, re-check `WindowsHello\Enabled` before re-investigating from scratch. It is a one-line fix, not a fresh diagnosis.
 - **So can any app you install.** Anything that wants WHP will enable it without asking and reset `hypervisorlaunchtype` to `Auto` as a side effect. Re-run step 03 after installing a new virtualization or emulator tool rather than assuming the host is still as you left it.
 - **Credential Guard and Memory Integrity stay off** as a consequence. On a lab machine that is an accepted trade; on a machine holding domain credentials, weigh it deliberately.
-

Written from the 2026-08-16 fix on the Ryzen 7 5800X desktop, where Windows Hello ESS was the sole VBS consumer and the two-command fix in 05b + 05c resolved it. Updated 2026-10-06 after Try Omarchy 0.9.0 re-enabled the hypervisor through the Windows Hypervisor Platform.

Diagnose before applying — the commands are the same, but the cause may not be.

Scripts that automate the diagnosis and the fix: github.com/brhoom98x/vmware-nested-virtualization-windows-11